

System Design — Master Checklist

Complete revision roadmap • Hinglish • Interview + Production focused

Goal: Is PDF ko final revision checklist ki tarah use karo. Har topic ke liye sirf definition nahi, balki architecture, bottleneck, failure handling, scaling aur trade-offs explain kar paana chahiye.

1. FOUNDATION

■ Topic	What you should be able to explain
Load Balancer	Traffic ko healthy backend instances mein distribute karta hai. L4/L7, health checks, algorithms, sticky sessions, stateless services, failover.
CDN / CloudFront	Static/media content ko edge locations par cache karke latency aur origin load reduce karta hai. Cache hit/miss, TTL, invalidation, signed URLs.
Redis	In-memory data store. Cache, session, rate limiting, distributed coordination, queues/streams jaise use cases. TTL, eviction, persistence, HA, hot keys.
Database Scaling	Slow queries → indexing → connection pool → vertical scaling → read replicas → partitioning/sharding. Bottleneck identify karke scale karna.
Indexing	Index query ko fast banata hai but writes/storage/maintenance cost badhata hai. Composite index, selectivity, EXPLAIN/query plan, covering index.
Replication / Read Replica	Primary writes, replicas reads. Replication lag, read-after-write consistency, failover, sync vs async replication.
Partitioning	Large table ko logical partitions mein split karna. Range/list/hash partitioning, partition pruning, hot partitions, archival.
Sharding	Data ko multiple DB nodes/shards mein distribute karna. Shard key, data skew, cross-shard query, rebalancing, routing.

2. DISTRIBUTED SYSTEMS

■ Topic	What you should be able to explain
Message Queue	Producer → Queue → Consumer. Async processing, spike buffering, retry, DLQ, ACK, visibility timeout, backpressure, idempotent consumers.
Rate Limiter	User/IP/API key ke request rate ko control karta hai. Fixed window, sliding window, token bucket, Redis-based distributed limiting, 429.
Notification	Event → Queue → channel workers → Email/SMS/Push providers. Retry, DLQ, provider limits, preferences, dedupe, delivery status.
Chat	WebSocket + connection management + message persistence + presence + ordering + offline sync. Multi-server ke liye shared messaging.

■ Topic	What you should be able to explain
Payment	Idempotency, payment states, gateway timeout = UNKNOWN, webhook verification, duplicate protection, refunds, reconciliation, ledger.
URL Shortener	Short code generation, Base62, collision handling, Redis cache, redirect path, analytics async, expiry, abuse controls.
File Upload	Presigned URL + object storage. Multipart upload, metadata DB, async processing, scanning, private files, CDN delivery.
Search	Inverted index, indexing pipeline, ranking, filters, autocomplete, typo tolerance, pagination, eventual consistency, reindexing.
API Gateway	Routing, authentication, rate limiting, validation, timeout/retry, circuit breaker, versioning, request ID, tracing.
Microservices	Business capability based service boundaries, database ownership, sync/async communication, service discovery, independent scaling/deployment.
CAP / Consistency	Partition ke time consistency vs availability trade-off. Strong/eventual consistency, read-after-write, quorum concepts.
Distributed Transactions	2PC vs Saga. Long distributed transactions avoid karna; local transactions + compensation, idempotency, failure recovery.
Saga / Outbox / Idempotency	Saga = distributed workflow; Outbox = DB change + event intent reliably capture; Idempotency = duplicate request/event se duplicate side effect prevent.

3. PRODUCTION

■ Topic	What you should be able to explain
Kubernetes	Pods, Deployments, Services, Ingress/Gateway, ConfigMap/Secret, probes, HPA, requests/limits, rolling/canary deployments, debugging.
Observability	System ke internal state ko logs, metrics aur traces se understand karna. Monitoring + debugging + SLO driven approach.
Prometheus / Grafana	Prometheus metrics scrape/store karta hai; Grafana dashboards/alerts visualize karta hai. PromQL, dashboards, alert thresholds.
Logs / Metrics / Traces	Logs = events/details; Metrics = numerical signals; Traces = request ka distributed journey. Correlation/trace IDs important.
p50 / p95 / p99	p50 typical latency; p95 tail latency; p99 extreme tail. Average slow requests ko hide kar sakta hai.
RPS / Capacity Estimation	DAU → requests/day → average RPS → peak multiplier → read/write split → storage → bandwidth → connection count.
Failure Handling	Detect → contain → recover → data corruption prevent → safe retry/replay → observe → prevent recurrence. Timeout, retry+jitter, circuit breaker, DLQ.

■ Topic	What you should be able to explain
Security	Authentication, authorization, TLS, secrets, least privilege, validation, rate limits, encryption, audit logs, secure file access.
Scaling	Vertical vs horizontal scaling, caching, replicas, partitioning, sharding, queues, stateless services, autoscaling, bottleneck-driven decisions.
SLO / SLA / RPO / RTO	SLO = internal reliability target; SLA = external/business commitment; RPO = acceptable data loss window; RTO = acceptable recovery time.

4. INTERVIEW

■ Topic	What you should be able to explain
Complete System Design Framework	Requirements → scale estimation → APIs/data model → high-level architecture → deep dive → scaling → consistency → failures → security → observability → trade-offs → summary.
Trade-offs	Har architecture decision ka benefit aur cost explain karo. 'Best' architecture mat bolo; requirement ke context mein choice justify karo.
Production Scenarios	Slow API, DB CPU high, Redis down, queue backlog, traffic spike, replica lag, WebSocket disconnects, duplicate payment, pod crash, external API timeout.
Golden Answers	Har answer mein: requirement clarify → quantify → bottleneck identify → design → failure handling → observability → trade-off → concise summary.

Final Interview Checklist

- Requirements: functional + non-functional requirements clarify kiye?
- Scale: DAU, RPS, peak RPS, read/write ratio estimate kiya?
- Data: entities, access patterns aur indexes define kiye?
- Architecture: client → edge → LB/Gateway → services → cache/queue → DB clearly explain kiya?
- Bottleneck: highest-risk component identify kiya?
- Caching: kya cache hoga, TTL/invalidation kya hoga?
- Async: kya queue/background processing useful hai?
- Consistency: strong ya eventual — aur kyun?
- Failure: timeout, retry, idempotency, circuit breaker, DLQ?
- Scaling: horizontal/vertical/replica/partition/shard decision?
- Security: auth, authorization, rate limit, secrets, encryption?
- Observability: logs, metrics, traces, p95/p99, alerts?
- Trade-offs: alternative aur cost explain ki?
- Final summary: 30–60 sec mein complete architecture repeat kar sakte ho?

45-Minute Interview Time Split

Time	Focus
0–5 min	Requirements + clarifying questions
5–10 min	Scale / capacity estimation
10–18 min	APIs + data model
18–25 min	High-level architecture
25–35 min	Deep dive into bottleneck / critical component
35–40 min	Failure handling + scaling + consistency
40–43 min	Security + observability
43–45 min	Trade-offs + final summary

Golden Questions

- Source of truth kya hai?
- Peak RPS kitna hai?
- Sabse hot path / bottleneck kya hai?
- Kaunsa operation synchronous rehna chahiye aur kya async ho sakta hai?
- Cache miss hone par kya hoga?
- DB slow/down hone par kya hoga?
- Duplicate request/event aaya to kya hoga?

- Traffic 10× ho gaya to kya scale karoge?
- Data consistency kitni strong chahiye?
- System unhealthy hai ye kaise detect karoge?

Final mindset: Ab goal aur topics collect karna nahi hai. Goal hai kisi bhi system ko requirements se architecture tak defend kar pana, aur production failure aane par bottleneck identify karke fix kar pana.